

Q1. Use Stream API to filter and print elements from a list.

```
import java.util.List;

public class StreamFilterDemo {
    public static void main(String[] args) {
        List<Integer> numbers = List.of(10, 15, 20, 25, 30, 35);

        numbers.stream()
            .filter(n -> n % 10 == 0)
            .forEach(System.out::println);
    }
}
```

The stream() method converts the list into a stream, filter keeps only the elements that satisfy the given condition (here, divisible by 10), and forEach prints each remaining element. This produces the output 10, 20, 30.

Q2. Write a lambda expression to compare two integers.

```
import java.util.Comparator;
import java.util.ArrayList;
import java.util.List;

public class LambdaComparatorDemo {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>(List.of(5, 2, 9, 1, 7));

        Comparator<Integer> comparator = (a, b) -> a - b;

        numbers.sort(comparator);
        System.out.println(numbers);
    }
}
```

The lambda (a, b) -> a - b implements the Comparator functional interface: it returns a negative number if a should come before b, zero if they are equal, and a positive number if a should come after b. Passing this lambda to sort arranges the list in ascending order, producing [1, 2, 5, 7, 9].

Q3. Chain map, filter, and collect operations in a Java program.

```
import java.util.List;
import java.util.stream.Collectors;

public class StreamChainDemo {
    public static void main(String[] args) {
        List<String> names = List.of("ravi", "anita", "sam", "priya", "jo");

        List<String> result = names.stream()
            .filter(name -> name.length() > 3)
            .map(String::toUpperCase)
            .collect(Collectors.toList());

        System.out.println(result);
    }
}
```

This chain first filters the list to keep only names with more than three characters (removing sam and jo), then maps each remaining name to its uppercase form, and finally collects the results back into a new list. The output is [RAVI, ANITA, PRIYA]. Chaining these operations in a single declarative pipeline is a hallmark of the functional style enabled by the Stream API.